COMSATS UNIVERSITY ISLAMABAD, ATTOCK CAMPUS

Department of Computer Sciences

Course Name: Information Security Lab

Assignment # 03

Submitted by: Muhammad Usama

Registration No: FA24-BSE-062

Instructor: Mam Ambreen Gul

Date: 04 May,2026

Blockchain Implementation

Code:

```
import hashlib
import datetime

# Block class
class Block:
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        value = str(self.index) + str(self.timestamp) + str(self.data) +
str(self.previous_hash)
        return hashlib.sha256(value.encode()).hexdigest()


# Blockchain class
class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, datetime.datetime.now(), "Genesis Block", "0")

    def get_last_block(self):
        return self.chain[-1]

    def add_block(self, data):
        last_block = self.get_last_block()
        new_block = Block(
            index=last_block.index + 1,
            timestamp=datetime.datetime.now(),
            data=data,
            previous_hash=last_block.hash
        )
        self.chain.append(new_block)

    def display_chain(self):
        for block in self.chain:
```

```python
            print("Index:", block.index)
            print("Timestamp:", block.timestamp)
            print("Data:", block.data)
            print("Previous Hash:", block.previous_hash)
            print("Hash:", block.hash)
            print("-" * 50)


# Create blockchain
my_blockchain = Blockchain()

# Add blocks
my_blockchain.add_block("Block 1 Data")
my_blockchain.add_block("Block 2 Data")
my_blockchain.add_block("Block 3 Data")
my_blockchain.add_block("Block 4 Data")

print("Original Blockchain:\n")
my_blockchain.display_chain()

# Modify block data
print("\nAfter modifying Block 2 data:\n")
my_blockchain.chain[2].data = "Hacked Data"
my_blockchain.chain[2].hash = my_blockchain.chain[2].calculate_hash()

my_blockchain.display_chain()
```

Output:

Original:

```
Index: 0
Timestamp: 2026-05-02 12:12:40.710943
Data: Genesis Block
Previous Hash: 0
Hash: a2b653cd3848ce6c8f41a250afcfcfec631d6d9cf60f3577a8fc8114d13f7f82
--------------------------------------------------
Index: 1
Timestamp: 2026-05-02 12:12:40.713088
Data: Block 1 Data
Previous Hash: a2b653cd3848ce6c8f41a250afcfcfec631d6d9cf60f3577a8fc8114d13f7f82
Hash: 06af269d83e9fca011255cb8ea7c374cbb5328a056fe19a2bc0bb04f82697142
--------------------------------------------------
Index: 2
Timestamp: 2026-05-02 12:12:40.713088
Data: Block 2 Data
Previous Hash: 06af269d83e9fca011255cb8ea7c374cbb5328a056fe19a2bc0bb04f82697142
Hash: 542495efeb613ef6fca0636f05626ce048ad57738ed0a1f48cc7514d9a2e8c49
--------------------------------------------------
Index: 3
Timestamp: 2026-05-02 12:12:40.713088
Data: Block 3 Data
Previous Hash: 542495efeb613ef6fca0636f05626ce048ad57738ed0a1f48cc7514d9a2e8c49
Hash: 6aa34d4f4ef68ee67bccd40f1587cf15986f5fc3c80ef5627ac7335814d6ee96
--------------------------------------------------
Index: 4
Timestamp: 2026-05-02 12:12:40.713088
Data: Block 4 Data
Previous Hash: 6aa34d4f4ef68ee67bccd40f1587cf15986f5fc3c80ef5627ac7335814d6ee96
Hash: 170b705b9e965b54395ae70e633f1e580352cffdd4101ccd5e90226e91ac01cd
--------------------------------------------------
```

After modifying:

```
Index: 0
Timestamp: 2026-05-02 12:12:40.710943
Data: Genesis Block
Previous Hash: 0
Hash: a2b653cd3848ce6c8f41a250afcfcfec631d6d9cf60f3577a8fc8114d13f7f82
--------------------------------------------------
Index: 1
Timestamp: 2026-05-02 12:12:40.713088
Data: Block 1 Data
Previous Hash: a2b653cd3848ce6c8f41a250afcfcfec631d6d9cf60f3577a8fc8114d13f7f82
Hash: 06af269d83e9fca011255cb8ea7c374cbb5328a056fe19a2bc0bb04f82697142
--------------------------------------------------
Index: 2
Timestamp: 2026-05-02 12:12:40.713088
Data: Hacked Data
Previous Hash: 06af269d83e9fca011255cb8ea7c374cbb5328a056fe19a2bc0bb04f82697142
Hash: 331f65ce59ccbcb8a086167905b40af86fbb1f4c7e02b84b30bc9d0307f51922
--------------------------------------------------
Index: 3
Timestamp: 2026-05-02 12:12:40.713088
Data: Block 3 Data
Previous Hash: 542495efeb613ef6fca0636f05626ce048ad57738ed0a1f48cc7514d9a2e8c49
Hash: 6aa34d4f4ef68ee67bccd40f1587cf15986f5fc3c80ef5627ac7335814d6ee96
--------------------------------------------------
Index: 4
Timestamp: 2026-05-02 12:12:40.713088
Data: Block 4 Data
Previous Hash: 6aa34d4f4ef68ee67bccd40f1587cf15986f5fc3c80ef5627ac7335814d6ee96
Hash: 170b705b9e965b54395ae70e633f1e580352cffdd4101ccd5e90226e91ac01cd
```

Code Explanation:

Introduction

In this program, a simple blockchain system is created using Python. The main idea behind this implementation is to understand how blocks are connected with each other and how security is maintained using hashing.

A blockchain is like a chain of boxes, where each box (block) stores some data and also keeps a reference to the previous box. This connection makes the system secure and reliable.

Libraries Used

```
import hashlib
import datetime
```

In this part:

- `hashlib` is used to create a secure digital fingerprint (hash)
- `datetime` is used to store the time when each block is created

Understanding the Block

```
class Block:
```

A block is the main component of the blockchain. It stores important information.

Each block contains:

- Index (position in chain)
- Timestamp (time of creation)

- Data (actual information)
- Previous hash (link to previous block)
- Hash (its own unique identity)

Block Initialization

When a block is created, all its values are assigned:

- The index tells us where the block is located
- Timestamp records when it was made
- Data stores the information
- Previous hash connects it to the previous block
- Hash is automatically generated

How Hash is Created

```
def calculate_hash(self):
```

Here:

- All block values are combined into a single string
- This string is passed through SHA-256
- A unique hash is generated

Even a small change in data will produce a completely different hash.

Blockchain Structure

```
class Blockchain:
```

This class manages the whole chain of blocks.

Genesis Block

The first block is called the **Genesis Block**.

```
self.chain = [self.create_genesis_block()]
```

- It is created automatically when the blockchain starts
- It has no previous block, so its previous hash is `"0"`

Adding New Blocks

When a new block is added:

1. The last block is taken
2. A new block is created
3. Its previous hash is set to the last block's hash
4. It is added to the chain

This ensures that all blocks are properly connected.

Displaying the Blockchain

The program prints each block's:

- Index
- Timestamp
- Data
- Previous hash
- Current hash

This helps in understanding how blocks are linked.

Testing Blockchain Security (Tampering)

```
my_blockchain.chain[2].data = "Hacked Data"
```

Here, we manually change the data of one block.

Then:

```
my_blockchain.chain[2].hash = my_blockchain.chain[2].calculate_hash()
```

- The hash of that block changes
- But the next block still contains the old hash

This breaks the chain!

Important Concept

Blockchain works because:

- Every block depends on the previous one
- Changing one block affects all the next blocks

This makes it **very hard to hack or modify data secretly**.

Security Understanding

- Data cannot be changed without detection
- The system is transparent
- It ensures trust between users

ECC (Elliptic Curve Cryptography)

Code:

```
# Elliptic Curve: y^2 = x^3 + ax + b (mod p)

p = 17   # prime number
a = 2
b = 2

# Define point addition
def point_add(P, Q):
    if P == Q:
        return point_double(P)

    x1, y1 = P
    x2, y2 = Q

    m = ((y2 - y1) * pow(x2 - x1, -1, p)) % p
    x3 = (m*m - x1 - x2) % p
    y3 = (m*(x1 - x3) - y1) % p

    return (x3, y3)


# Point doubling
def point_double(P):
    x, y = P
```

```python
    m = ((3*x*x + a) * pow(2*y, -1, p)) % p
    x3 = (m*m - 2*x) % p
    y3 = (m*(x - x3) - y) % p

    return (x3, y3)


# Scalar multiplication (kP)
def scalar_mult(k, P):
    result = P
    for i in range(k - 1):
        result = point_add(result, P)
    return result


# Base point
G = (5, 1)

# Private key
d = 3

# Public key
Q = scalar_mult(d, G)

print("Private Key:", d)
print("Public Key:", Q)


# Encryption (simplified EC-ElGamal)
message = 7

k = 2  # random number

C1 = scalar_mult(k, G)
C2 = message + scalar_mult(k, Q)[0]

print("\nEncrypted:")
print("C1:", C1)
print("C2:", C2)


# Decryption
decrypted = C2 - scalar_mult(d, C1)[0]

print("\nDecrypted Message:", decrypted)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\samoo\Desktop\IS lab> & C:/Users/samoo/Ap
s/samoo/Desktop/IS lab/ECC.py"
Private Key: 3
Public Key: (10, 6)

Encrypted:
C1: (6, 3)
C2: 23

Decrypted Message: 7
PS C:\Users\samoo\Desktop\IS lab>
```

Explanation:

Overview

In this task, Elliptic Curve Cryptography (ECC) is implemented using small numerical values to understand its working. ECC is a modern encryption technique that uses points on a mathematical curve instead of large prime factorization.

Elliptic Curve Formula

The equation used in this program is:

```
y^2 = x^3 + ax + b (mod p)
```

This equation defines all valid points on the curve under modular arithmetic.

Curve Initialization

```
p = 17
a = 2
b = 2
```

- A small prime number `p` is selected to keep calculations simple
- Values `a` and `b` determine the shape of the curve
- All operations are performed modulo `p`

Function 1: Point Addition

```
def point_add(P, Q):
```

This function is used to add two different points on the elliptic curve.

- If both input points are identical, the function redirects to point doubling

- Coordinates of both points are separated into `(x1, y1)` and `(x2, y2)`
- A slope is calculated using modular inverse
- New coordinates `(x3, y3)` are computed using ECC formulas
- The resulting point is returned

Function 2: Point Doubling

```
def point_double(P):
```

This function handles the case when the same point is added to itself.

- Extracts the coordinates `(x, y)`
- Computes slope using a different formula compared to point addition
- Calculates new point `(x3, y3)`
- Returns the doubled point

Function 3: Scalar Multiplication

```
def scalar_mult(k, P):
```

This function performs repeated addition of a point.

- Starts with the initial point `P`
- Adds the point to itself `k-1` times
- Returns the final result, which represents `k × P`

Base Point Selection

```
G = (5, 1)
```

- This point acts as the **generator point**
- It is publicly known and used in key generation

Key Generation Process

Private Key

```
d = 3
```

- A secret integer chosen by the user

Public Key

```
Q = scalar_mult(d, G)
```

- Public key is obtained by multiplying the base point with private key

Encryption Phase

Message Definition

```
message = 7
```

- A small numeric message is selected for demonstration

Random Value

```
k = 2
```

- A random integer used to increase security

Cipher Calculation

```
C1 = scalar_mult(k, G)
C2 = message + scalar_mult(k, Q)[0]
```

- `C1` is generated using base point multiplication
- `C2` combines message with the public key
- Together `(C1, C2)` form the encrypted data

Decryption Process

```
decrypted = C2 - scalar_mult(d, C1)[0]
```

- The receiver uses their private key to recover the message
- The original message is successfully retrieved

Important Concepts

- ECC works with **points instead of numbers**
- Security depends on difficulty of reversing scalar multiplication
- Even with small values, the logic of real-world ECC is demonstrated

Security Perspective

- ECC is highly secure due to the **Elliptic Curve Discrete Logarithm Problem**
- It provides strong encryption using smaller keys
- Commonly used in:
    - Secure websites (HTTPS)
    - Digital signatures
    - Cryptographic systems